



MPJ Express



MPJ-Express es una implementación en Java del estándar **MPI** (Message Passing Interface), destinado a la programación concurrente distribuida basada en **paso de mensajes**. En MPI, cada proceso tiene **su propia memoria privada**, y la comunicación entre procesos se realiza exclusivamente enviando y recibiendo mensajes explícitos

Explicación

MPJ-Express traslada la filosofía de MPI al ecosistema Java, pero **el modelo conceptual es idéntico**:

- No hay hilos → **hay procesos independientes**.
- No hay memoria compartida → **cada proceso tiene su propio espacio de direcciones**. Por este motivo no tiene mucho sentido hablar de hilos.
- La comunicación se hace mediante **send/receive**, nunca mediante variables compartidas.

Esto es clave para evitar confusiones con Java multihilo, que sí usa memoria compartida.

Toda interacción entre procesos se hace mediante mensajes explícitos.

Inicialización y ejecución básica

En MPJ, cada proceso es una **instancia independiente del programa**, creada por el lanzador MPJ.

Las funciones esenciales son:

```
MPI.Init(args);      // Inicializa el entorno
MPI.COMM_WORLD.Size(); // Número de procesos en el comunicador
MPI.COMM_WORLD.Rank(); // Identificador (0..size-1) del proceso actual. Los
                        // identificadores de procesos empiezan en 0 y se van incrementando en 1.
MPI.Finalize();      // Finaliza el entorno
```

COMM_WORLD

Es el **comunicador por defecto**, que engloba a TODOS los procesos lanzados.

Un comunicador define:

- Qué procesos se incluyen.
- Qué comunicaciones tienen sentido.
- Qué identificador (`rank`) tiene cada proceso dentro de ese grupo.

Los comunicadores permiten **crear subgrupos independientes**, de forma que distintas partes de una aplicación (o bibliotecas internas) pueden comunicarse sin interferirse entre sí. No hay un tamaño fijo de comunicadores que podamos crear, pueden ser los que sean.

👉 Un proceso puede tener **distintos ranks** en distintos comunicadores.

Cuando ejecutemos alguna operación de MPJ-Express, es MUY IMPORTANTE entender qué operaciones van sobre el comunicador y cuáles no, ya que con esto nos la pueden liar en el examen.

Por ejemplo, `MPI.COMM_WORLD.Size()` es una operación en la que hay que especificar un comunicador. Nos dará el número de procesos de ESE COMUNICADOR, no de otro.

Comunicación Punto a Punto (P2P)

En MPJ-Express, `Send()` y `Recv()` son **bloqueantes y síncronas**:

- `Send()` bloquea hasta que el mensaje es seguro (entregado o copiado a un buffer interno).
- `Recv()` bloquea hasta que se recibe el mensaje.

Ambas funciones requieren especificar:

- El buffer,
- El desplazamiento,
- El número de elementos,
- El tipo (ej: `MPI.INT`),
- El `rank` del otro proceso,
- El `tag` (un identificador de mensaje).

Ejemplo básico:

```
import mpi.*;

public class EjemploP2P {
    public static void main(String[] args) {
        MPI.Init(args);

        int rank = MPI.COMM_WORLD.Rank();

        if (rank == 0) {
            int[] dato = {42};
            MPI.COMM_WORLD.Send(dato, 0, 1, MPI.INT, 1, 0);
        }
        else if (rank == 1) {
            int[] buffer = new int[1];
            MPI.COMM_WORLD.Recv(buffer, 0, 1, MPI.INT, 0, 0);
            System.out.println("Proceso 1 recibió: " + buffer[0]);
        }

        MPI.Finalize();
    }
}
```

Tipos de datos y *Status*

MPI exige declarar explícitamente el tipo de dato para que los procesos puedan interpretar correctamente la comunicación.

Es programación distribuida, así que la arquitectura del procesador que tenga la máquina receptora no tiene que ser la misma que la de la máquina emisora, así que hay que indicar el tipo de dato.

Al recibir un mensaje, podemos usar el objeto `Status` para obtener información como:

- Tamaño real del mensaje,
- Remitente,
- Tag recibido.

¿Cuántos procesos ejecuta MPJ-Express?

Los que quiera.

Podemos obtener el número con la operación `size()`, pero nunca podemos tener la certeza de que será un número específico.

Lo único que podemos tener claro es que el número de procesos MÁXIMO será el que le pasemos como argumento cuando lanzamos el programa, pero no tiene que ser justo el mismo. Puede ser el mismo o menor, en función de lo que MPJ-Express quiera.

Incluso si el programador solicita 16 procesos, el entorno puede decidir lanzar menos en función de los recursos disponibles.

Por eso:

👉 **`size()` puede ser menor que el número solicitado.**

Comunicación Colectiva

A diferencia de P2P, la comunicación colectiva **no requiere especificar procesos destino**:

el conjunto de participantes lo define el **comunicador**.

Principales operaciones:

`MPI_Barrier`

Sincroniza todos los procesos del comunicador.

```
import mpi.*;

public class EjemploBarrier {
    public static void main(String[] args) throws Exception {
        MPI.Init(args);

        int rank = MPI.COMM_WORLD.Rank();

        System.out.println("[Barrier] Proceso " + rank + " antes de la barrera");

        // Punto de sincronización global
        MPI.COMM_WORLD.Barrier();

        System.out.println("[Barrier] Proceso " + rank + " después de la barrera");

        MPI.Finalize();
    }
}
```

MPI_Bcast

Un proceso (root) envía un dato a todos los demás.

```
import mpi.*;

public class EjemploBroadcast {
    public static void main(String[] args) throws Exception {
        MPI.Init(args);

        int rank = MPI.COMM_WORLD.Rank();
        int[] dato = new int[1];

        if (rank == 0) {
```

```

        dato[0] = 42;
        System.out.println("[Bcast] Root (0) va a enviar: " + dato[0]);
    }

    // Root = 0 difunde dato[0] a todos
    MPI.COMM_WORLD.Bcast(dato, 0, 1, MPI.INT, 0);

    System.out.println("[Bcast] Proceso " + rank + " recibió: " + dato[0]);

    MPI.Finalize();
}
}

```

Ejemplo:

```

import mpi.*;

public class EjemploBroadcast {
    public static void main(String[] args) {
        MPI.Init(args);
        int rank = MPI.COMM_WORLD.Rank();

        int[] dato = new int[1];
        if (rank == 0) dato[0] = 99;

        MPI.COMM_WORLD.Bcast(dato, 0, 1, MPI.INT, 0);

        System.out.println("Proceso " + rank + " recibió " + dato[0]);

        MPI.Finalize();
    }
}

```

MPI_Scatter

Reparte partes **distintas** de un buffer del root a cada proceso.

- P0 recibe `sendbuf[0]`
- P1 recibe `sendbuf[1]`
- P2 recibe `sendbuf[2]`
- ...

No se indican "destinos": lo determina el comunicador.

```
import mpi.*;

public class EjemploScatter {
    public static void main(String[] args) throws Exception {
        MPI.Init(args);

        int rank = MPI.COMM_WORLD.Rank();
        int size = MPI.COMM_WORLD.Size();

        int[] sendbuf = null;
        int[] recvbuf = new int[1]; // cada proceso recibe 1 entero

        if (rank == 0) {
            sendbuf = new int[size];
            for (int i = 0; i < size; i++) {
                sendbuf[i] = (i + 1) * 10; // 10,20,30,...
            }

            System.out.print("[Scatter] Root tiene: ");
            for (int v : sendbuf) System.out.print(v + " ");
            System.out.println();
        }

        MPI.COMM_WORLD.Scatter(
            sendbuf, 0, 1, MPI.INT, // origen (root)
            recvbuf, 0, 1, MPI.INT, // destino (cada proceso)
            0 // root
        );
    }
}
```

```

);

System.out.println("[Scatter] Proceso " + rank +
    " recibió: " + recvbuf[0]);

MPI.Finalize();
}
}

```

MPI_Gather

Operación inversa: recolecta datos de todos los procesos en uno solo.

```

import mpi.*;

public class EjemploGather {
    public static void main(String[] args) throws Exception {
        MPI.Init(args);

        int rank = MPI.COMM_WORLD.Rank();
        int size = MPI.COMM_WORLD.Size();

        int[] sendbuf = new int[1];
        sendbuf[0] = rank * 100; // cada proceso manda algo distinto

        int[] recvbuf = null;
        if (rank == 0) {
            recvbuf = new int[size]; // root recoge size valores
        }

        MPI.COMM_WORLD.Gather(
            sendbuf, 0, 1, MPI.INT, // lo que envía cada proceso
            recvbuf, 0, 1, MPI.INT, // lo que recibe root
            0 // root
        );
    }
}

```



```

    if (rank == 0) {
        System.out.print("[Gather] Root ha recogido: ");
        for (int v : recvbuf) System.out.print(v + " ");
        System.out.println();
    }

    MPI.Finalize();
}
}

```

MPI_Reduce

Realiza una operación (sum, max, min...) sobre datos distribuidos.

```

import mpi.*;

public class EjemploReduce {
    public static void main(String[] args) throws Exception {
        MPI.Init(args);

        int rank = MPI.COMM_WORLD.Rank();
        int size = MPI.COMM_WORLD.Size();

        int[] sendbuf = new int[1];
        sendbuf[0] = rank; // cada proceso aporta su rank

        int[] recvbuf = new int[1]; // root recibirá el resultado

        MPI.COMM_WORLD.Reduce(
            sendbuf, 0,      // dato local
            recvbuf, 0,      // resultado en root
            1, MPI.INT,      // 1 entero
            MPI.SUM,         // operación: suma
            0                 // root
        );
    }
}

```

```

    if (rank == 0) {
        int esperado = size * (size - 1) / 2;
        System.out.println("[Reduce] Suma de todos los ranks = " + recvbuf
[0]);
        System.out.println("[Reduce] Valor esperado      = " + esperado);
    }

    MPI.Finalize();
}
}

```

MPI_Scan

Reducción de prefijos.

```

import mpi.*;

public class EjemploScan {
    public static void main(String[] args) throws Exception {
        MPI.Init(args);

        int rank = MPI.COMM_WORLD.Rank();

        int[] sendbuf = new int[1];
        int[] recvbuf = new int[1];

        sendbuf[0] = 1; // todos aportan 1

        // Prefijo de suma: cada proceso recibe suma de [0..rank]
        MPI.COMM_WORLD.Scan(
            sendbuf, 0,
            recvbuf, 0,
            1, MPI.INT,
            MPI.SUM
        );
    }
}

```

```
System.out.println("[Scan] Proceso " + rank +  
    " → suma prefijo hasta aquí = " + recvbuf[0]);  
  
MPI.Finalize();  
}  
}
```

Ejecución de un programa MPJ-Express

Para ejecutar:

1. **Iniciar el servicio de nombres (DNS interno de MPJ).**
Sin él, los procesos no pueden localizarse.
2. **Lanzar el servidor MPJ.**
3. **Ejecutar los procesos cliente.**